SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Question 1: Resume Information Extraction

Aim

Extract candidate details and shortlist eligible candidates using Regex.

Objectives

- Extract name, email, phone, skills, experience.
- Generate summary.
- Shortlist candidates with Python and >=2 years experience.

Algorithm

1. Read resume.
2. Extract fields using regex.
3. Generate summary.
4. Check eligibility.
5. Display eligible candidates.

Python Program

```
import re

# Sample resumes

resumes = [

"""

John Smith

Email: johnsmith@gmail.com

Phone: +91-9876543210

Skills: Python, SQL, Machine Learning, NLP

Experience: 3 years

""",
```

```
"""
Priya Sharma
Email: priya@gmail.com
Mobile: 9123456789
Skills: Java, SQL
Experience: 1 year
""",

"""
Rahul Kumar
Email: rahul123@yahoo.com
Contact: 9876512345
Skills: Python, Java, SQL
Experience: 5 years
"""
]
# Technical skills list
skill_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
def extract_information(text):

    # Name (first line)
    name = text.strip().split('\n')[0]
    # Email
    email = re.findall(r'[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}', text)
    # Phone Number
    phone = re.findall(r'(\+91[- ]?)?[6-9]\d{9}', text)
    # Skills
    skills_found = []
    for skill in skill_list:
```

```
        if re.search(skill, text, re.IGNORECASE):
            skills_found.append(skill)
    # Experience
    experience = re.search(r'(\d+)\s+year', text, re.IGNORECASE)
    years = int(experience.group(1)) if experience else 0
    return {
        "Name": name,
        "Email": email[0] if email else "Not Found",
        "Phone": phone[0] if phone else "Not Found",
        "Skills": skills_found,
        "Experience": years
    }
eligible = []
print("----------- Candidate Summary -----------\n")
for resume in resumes:
    candidate = extract_information(resume)
    print("Name :", candidate["Name"])
    print("Email :", candidate["Email"])
    print("Phone :", candidate["Phone"])
    print("Skills :", ", ".join(candidate["Skills"]))
    print("Experience :", candidate["Experience"], "Years")
    if candidate["Experience"] >= 2 and "Python" in candidate["Skills"]:
        eligible.append(candidate)
    print("-------------------------------------")
print("\nEligible Candidates\n")
for person in eligible:
    print(person["Name"], "-", person["Experience"], "Years")
```

Conclusion

Regex enables automatic resume parsing and efficient candidate shortlisting.

Question 2: Product Search System

Aim

Develop a product search engine using Regular Expressions.

Objectives

- Exact keyword search
- Prefix search
- Suffix search
- Partial keyword search
- Case-insensitive search
- Display matches and total count

Algorithm

6. Store product names.
7. Accept search keyword.
8. Build regex based on search type.
9. Display matching products.
10. Generate search report.

Python Program

```
import re
products=["Python Book","Java Programming","SQL Guide","Machine Learning Kit",
"NLP Toolkit","Python Mug","Smart Phone","Phone Cover","Laptop Stand"]

def search_products(keyword, mode):
    if mode=="exact":
        pattern=f"^{re.escape(keyword)}$"
    elif mode=="prefix":
        pattern=f"^{re.escape(keyword)}"
    elif mode=="suffix":
        pattern=f"{re.escape(keyword)}$"
    elif mode=="partial":
        pattern=re.escape(keyword)
    result=[p for p in products if re.search(pattern,p,re.IGNORECASE)]
    print("Matches:",result)
    print("Total:",len(result))

search_products("Python","prefix")
search_products("Guide","suffix")
search_products("Phone","partial")
search_products("python book","exact")
```

Conclusion

The system improves product discovery by supporting flexible keyword matching.

Question 3: Student Registration Validation System

Aim

Validate student registration details using Regular Expressions.

Objectives

- Validate register number
- Validate institutional email
- Validate course code
- Validate semester
- Validate mobile number
- Generate registration status

Algorithm

11. Read inputs.
12. Validate each field using regex.
13. Display validation messages.
14. Generate final registration report.

Python Program

```
import re
reg_no="23CSE1234"
email="student@university.edu"
course="CSE301"
semester="Semester 5"
mobile="9876543210"

print("Register No:",bool(re.fullmatch(r"\d{2}[A-Z]{3}\d{4}",reg_no)))
print("Email:",bool(re.fullmatch(r"[A-Za-z0-9._%+-]+@university\.edu",email)))
print("Course:",bool(re.fullmatch(r"[A-Z]{3}\d{3}",course)))
print("Semester:",bool(re.fullmatch(r"Semester\s([1-8])",semester)))
print("Mobile:",bool(re.fullmatch(r"[6-9]\d{9}",mobile)))

if all([
re.fullmatch(r"\d{2}[A-Z]{3}\d{4}",reg_no),
re.fullmatch(r"[A-Za-z0-9._%+-]+@university\.edu",email),
re.fullmatch(r"[A-Z]{3}\d{3}",course),
re.fullmatch(r"Semester\s([1-8])",semester),
re.fullmatch(r"[6-9]\d{9}",mobile)]):
    print("Registration Successful")
else:
    print("Registration Failed")
```

Conclusion

Regex-based validation ensures that only correctly formatted student details are accepted.